

Sistema de Gestión de Instalaciones UHU

Memoria Técnica del Proyecto

Autor: Agustín Rodríguez Aguilar

Asignatura: Desarrollo de Aplicaciones Web

Curso: 2025-2026

Repositorio GitHub:

[instalaciones-uhu](#)

Web Personal:

[agustinrodriguez.com](#)

Universidad de Huelva

Escuela Técnica Superior de Ingeniería

30 de noviembre de 2025

Resumen

El proyecto **Instalaciones UHU** representa una solución desarrollada en Jakarta EE para la gestión centralizada de las instalaciones deportivas de la Universidad de Huelva. Esta aplicación web aborda la problemática de la gestión descentralizada de espacios deportivos mediante una plataforma unificada que permite la administración eficiente de usuarios, instalaciones y reservas.

La arquitectura del sistema se fundamenta en el patrón Modelo-Vista-Controlador (MVC), implementado mediante Servlets como controladores, JSP para la capa de presentación y modelos Java para la lógica de negocio. El proyecto destaca por su robustez en la gestión de seguridad, implementando autenticación con hash de contraseñas, control de sesiones y autorización basada en roles.

Desarrollado en el IDE NetBeans con el sistema de build Apache Ant y desplegado en GlassFish Server, la aplicación incorpora características avanzadas como validación de datos en cliente y servidor, gestión de disponibilidad en tiempo real e integración potencial con sistemas de pago mediante Stripe.

Índice general

Resumen	1
1. Introducción y Justificación	4
1.1. Contexto y Motivación	4
1.2. Propósito del Sistema	4
1.3. Alcance del Problema	4
2. Objetivos y Alcance del Proyecto	5
2.1. Objetivo General	5
2.2. Objetivos Específicos Técnicos	5
2.2.1. Arquitectura y Desarrollo	5
2.2.2. Funcionalidades Core	5
2.2.3. Seguridad y Usabilidad	5
2.3. Alcance Funcional Detallado	6
2.3.1. Incluido en el Alcance	6
2.3.2. Excluido del Alcance Inicial	6
3. Análisis Técnico y Arquitectura	7
3.1. Arquitectura del Sistema	7
3.1.1. Patrón MVC Implementado	7
3.1.2. Flujo de Peticiones	7
3.2. Estructura Detallada del Proyecto	8
3.3. Tecnologías y Versiones	9
3.3.1. Backend y Servidor	9
3.3.2. Frontend y Cliente	9
3.3.3. Dependencias Externas	9
3.3.4. Herramientas de Desarrollo	9
4. Modelo de Datos y Entidades	10
4.1. Análisis de los modelos (Entity)	10
4.1.1. Entidad: Usuario	10
4.1.2. Entidad: Espacio Deportivo	11
4.1.3. Entidad: Reserva	11
5. Implementación y Desarrollo	13
5.1. Análisis de los Controladores (Servlets)	13
5.1.1. Controlador de Usuarios	13
5.1.2. Controlador de Espacios Deportivos	15
5.1.3. Controlador de Reservas	17

6. Capa de Presentación	20
6.1. Arquitectura de Vistas	20
6.1.1. Estructura de Directorios	20
6.1.2. Mecanismo de Inyección Dinámica	20
6.1.3. Lógica de Presentación (JSTL)	21
6.1.4. Integración de Recursos Estáticos	21
7. Lógica de Cliente (Scripts)	22
7.1. Arquitectura de Ficheros	22
7.2. Análisis de Componentes	22
7.2.1. Validación Híbrida (<code>validacion-formularios.js</code>)	22
7.2.2. Gestión de Interfaz (<code>index.js</code>)	23
7.2.3. Control de Sesión (<code>logout.js</code>)	23
8. Configuración y Despliegue	24
8.1. Requisitos del Sistema	24
8.1.1. Requisitos Mínimos	24
8.1.2. Requisitos Recomendados	24
8.2. Configuración del Entorno	25
8.2.1. Configuración NetBeans	25
8.2.2. Configuración GlassFish	25
8.3. Proceso de Despliegue	25
8.3.1. Despliegue en Desarrollo	25
8.3.2. Despliegue en Producción	26
8.4. Configuración de Base de Datos	26
9. Funcionalidades del Sistema	27
9.1. Módulo de Autenticación y Autorización	27
9.1.1. Registro de Usuarios	27
9.1.2. Inicio de Sesión	27
9.1.3. Gestión de Perfiles	27
9.2. Módulo de Gestión de Instalaciones	28
9.2.1. CRUD de Espacios Deportivos	28
9.2.2. Gestión de Imágenes	28
9.3. Módulo de Reservas	29
9.3.1. Creación de Reservas	29
9.3.2. Gestión de Disponibilidad	29
9.3.3. Historial y Gestión	29
9.4. Panel Administrativo	30
9.4.1. Gestión de Usuarios	30
9.4.2. Administración del Sistema	30
9.5. Características de Seguridad	30
9.5.1. Protección de Datos	30
9.5.2. Control de Acceso	30
9.6. Conclusión Final	31

Capítulo 1

Introducción y Justificación

1.1. Contexto y Motivación

La Universidad de Huelva, como institución educativa de referencia, cuenta con numerosas instalaciones deportivas distribuidas por el campus. La gestión tradicional de estos espacios mediante procesos manuales y sistemas que desde mi punto de vista, no están bien desarrollados y gestionados, esto generaba para mi importantes desafíos a resolver:

- **Fragmentación de información:** Datos dispersos en diferentes departamentos
- **Ineficiencia en reservas:** Procesos manuales propensos a errores y conflictos de horarios
- **Falta de visibilidad:** Dificultad para conocer la disponibilidad real de instalaciones
- **Experiencia de usuario deficiente:** Múltiples puntos de contacto y procedimientos inconsistentes

1.2. Propósito del Sistema

El sistema **Instalaciones UHU** se concibe como una plataforma unificada que centraliza toda la gestión de instalaciones deportivas universitarias. Su propósito fundamental es optimizar la utilización de recursos, mejorar la experiencia de usuarios (estudiantes, personal y administradores) y proporcionar herramientas modernas para la administración eficiente de espacios deportivos.

1.3. Alcance del Problema

La solución aborda problemáticas específicas identificadas en el contexto universitario:

- Gestión de accesos y permisos diferenciados por roles
- Control de disponibilidad y prevención de conflictos de reservas
- Mantenimiento centralizado del catálogo de instalaciones
- Generación de historiales y reportes de utilización
- Garantía de seguridad en el manejo de datos sensibles

Capítulo 2

Objetivos y Alcance del Proyecto

2.1. Objetivo General

Desarrollar e implementar un sistema web robusto, escalable y seguro para la gestión integral de instalaciones deportivas de la Universidad de Huelva, que sirva como plataforma única para todos los procesos relacionados con la administración y uso de estos espacios.

2.2. Objetivos Específicos Técnicos

2.2.1. Arquitectura y Desarrollo

1. Implementar una arquitectura MVC utilizando tecnologías Jakarta EE estándar
2. Diseñar un sistema modular que permita el mantenimiento y escalabilidad
3. Garantizar la compatibilidad con servidores GlassFish y estándares Jakarta EE
4. Establecer una estructura de proyecto organizada y documentada

2.2.2. Funcionalidades Core

1. Desarrollar sistema completo de autenticación y autorización
2. Implementar CRUD completo para gestión de los distintos modelos dentro de la aplicación
3. Crear motor de reservas con validación de disponibilidad en tiempo real
4. Diseñar interfaces administrativas para gestión del sistema

2.2.3. Seguridad y Usabilidad

1. Implementar medidas de seguridad robustas (hash, sesiones, validaciones)
2. Garantizar la protección contra vulnerabilidades web comunes
3. Diseñar interfaces intuitivas y responsive
4. Asegurar la consistencia en la experiencia de usuario

2.3. Alcance Funcional Detallado

2.3.1. Incluido en el Alcance

- **Sistema de Autenticación:**
 - Registro y validación de nuevos usuarios
 - Login seguro con gestión de sesiones
 - Roles diferenciados (usuario/estudiante, profesor/personal, administrador)
 - Logout controlado
- **Gestión de Instalaciones:**
 - Catálogo completo de espacios deportivos
 - CRUD de instalaciones (solo administradores)
 - Gestión de imágenes y detalles técnicos
- **Sistema de Reservas:**
 - Consulta de disponibilidad en tiempo real
 - Creación de reservas con validaciones
 - Gestión de conflictos de horarios
 - Historial personal de reservas
- **Panel Administrativo:**
 - Gestión completa de usuarios
 - Administración centralizada de instalaciones
 - Monitorización de reservas activas

2.3.2. Excluido del Alcance Inicial

- Sistema de pagos online integrado (aunque se incluyen librerías de Stripe)
- Aplicación móvil nativa
- Integración con sistemas académicos existentes
- Módulo avanzado de reportes y analytics
- Sistema de notificaciones push o por email
- Filtrado de elementos del modelo

Capítulo 3

Análisis Técnico y Arquitectura

3.1. Arquitectura del Sistema

3.1.1. Patrón MVC Implementado

El proyecto sigue estrictamente el patrón Modelo-Vista-Controlador, con una separación clara de responsabilidades:

- **Modelo (Model):** Clases Java que representan entidades y lógica de negocio
 - Entidades: Usuario.java, EspacioDeportivo.java, Reserva.java
 - Servicios: AuthService.java
 - Lógica de validación y reglas de negocio
- **Vista (View):** Páginas JSP que renderizan la interfaz de usuario
 - Templates reutilizables (layout.jsp)
 - Vistas específicas por módulo
 - Componentes de UI modulares
- **Controlador (Controller):** Servlets que manejan las peticiones HTTP
 - controladorUsuario.java
 - controladorEspacioDeportivo.java
 - controladorReserva.java

3.1.2. Flujo de Peticiones

El flujo típico de una petición en el sistema es:

1. El usuario realiza una acción en la vista (JSP)
2. La petición HTTP llega al Servlet correspondiente
3. El Servlet procesa los parámetros y valida la solicitud
4. Se invocan los servicios y modelos necesarios
5. El Servlet decide qué vista mostrar como respuesta
6. La vista JSP se renderiza con los datos proporcionados

3.2. Estructura Detallada del Proyecto

Listing 3.1: Estructura completa del proyecto

```

1 instalaciones-uhu/
2 |-- src/
3 |   |-- java/
4 |     |-- app/
5 |       |-- controladores/                # Capa Controlador
6 |         |-- controladorUsuario.java
7 |         |-- controladorEspacioDeportivo.java
8 |         |-- controladorReserva.java
9 |       |-- modelos/                      # Capa Modelo
10 |        |-- Usuario.java
11 |        |-- EspacioDeportivo.java
12 |      |-- servicios/                     # Logica de negocio
13 |        |-- AuthService.java
14 |-- web/
15 |   |-- WEB-INF/
16 |     |-- vistas/                        # JSPs - Capa Vista
17 |       |-- auth/
18 |         |-- login.jsp
19 |         |-- registro.jsp
20 |       |-- admin/
21 |         |-- panelUsuarios.jsp
22 |         |-- editarUsuario.jsp
23 |       |-- instalaciones/
24 |         |-- panelInstalaciones.jsp
25 |         |-- listaInstalaciones.jsp
26 |         |-- detalleInstalacion.jsp
27 |         |-- formInstalacion.jsp
28 |       |-- reservas/
29 |         |-- listaReservas.jsp
30 |         |-- disponibilidadInstalacion.jsp
31 |       |-- templates/
32 |         |-- layout.jsp                 # Template base
33 |         |-- error.jsp
34 |   |-- lib/                             # Dependencias
35 |   |-- styles/
36 |     |-- index.css
37 |     |-- usuarios.css
38 |     |-- reservas.css
39 |     |-- espacios.css
40 |   |-- scripts/
41 |     |-- index.js
42 |     |-- registro.js
43 |     |-- validacion-formularios.js
44 |     |-- logout.js
45 |   |-- img/
46 |     |-- instalaciones/                 # Imagenes
47 |     |-- index.html

```

3.3. Tecnologías y Versiones

3.3.1. Backend y Servidor

- **Java:** JDK 8 o superior
- **Jakarta EE:** Versión 7 o 8 (compatible con GlassFish)
- **Servlets:** API 3.1 o superior
- **JSP:** Versión 2.3 o superior
- **GlassFish Server:** Versión 5.x o superior

3.3.2. Frontend y Cliente

- **HTML5:** Estructura semántica
- **CSS3:** Estilos y diseño responsive
- **JavaScript (ES6+):** Interactividad del cliente
- **JSP (JavaServer Pages):** Generación dinámica de contenido

3.3.3. Dependencias Externas

- **Stripe Java (24.9.0):** Integración con pasarela de pagos

3.3.4. Herramientas de Desarrollo

- **NetBeans IDE:** Entorno de desarrollo integrado
- **Apache Ant:** Sistema de build y automatización
- **Git:** Control de versiones
- **GlassFish:** Servidor de aplicaciones para desarrollo

Capítulo 4

Modelo de Datos y Entidades

4.1. Análisis de los modelos (Entity)

La capa de persistencia se ha implementado utilizando el estándar **Jakarta Persistence API (JPA)**. El modelo de dominio se compone de tres entidades principales que mapean las estructuras relacionales de la base de datos, garantizando la integridad referencial y facilitando la manipulación de datos mediante objetos Java (POJOs).

Todas las entidades implementan la interfaz **Serializable** para permitir su transferencia a través de la red o su almacenamiento en sesiones HTTP distribuidas si fuera necesario.

4.1.1. Entidad: Usuario

La clase **Usuario** representa a los actores del sistema. Además de almacenar la información personal, actúa como el núcleo del sistema de Control de Acceso Basado en Roles (RBAC).

Características técnicas:

- **Restricciones de Unicidad:** Se aplican restricciones `unique = true` a nivel de base de datos en los campos `dni` y `email` para garantizar la consistencia de datos críticos.
- **Gestión de Roles:** Se utiliza un discriminador numérico (`int`) para la jerarquía de permisos:
 - 0: Administrador (Acceso total).
 - 1: Estudiante (Acceso estándar con tarifas base).
 - 2: Profesor (Acceso con bonificación de tarifas).
- **Máquina de Estados:** El campo `solicitudProfesor` implementa una pequeña máquina de estados (`NULL` → `PENDIENTE` → `APROBADA/RECHAZADA`) para gestionar el flujo de ascenso de privilegios.

Atributo	Tipo Java	Descripción
id	Long	Clave primaria (Auto-increment).
dni	String	Identificador legal único.
nombre	String	Nombre completo del usuario.
email	String	Credencial de acceso (Login).
password	String	Hash MD5 de la contraseña.
rol	int	Nivel de autorización.

Cuadro 4.1: Diccionario de datos: Entidad Usuario

4.1.2. Entidad: Espacio Deportivo

La clase `EspacioDeportivo` modela los recursos físicos reservables. Esta entidad está desacoplada de la lógica de reservas, actuando puramente como un catálogo de inventario.

Características técnicas:

- **Persistencia de Recursos:** El campo `imagenUrl` almacena una ruta relativa (ej. `/img/instalaciones/temp/...`), desacoplando la base de datos de la ubicación física de los archivos en el servidor.
- **Igualdad Semántica:** Sobrescribe los métodos `equals()` y `hashCode()` basándose únicamente en el `id`, lo que es crucial para evitar duplicados cuando se manejan colecciones de espacios en memoria (ej. `Set<EspacioDeportivo>`).

Atributo	Tipo Java	Descripción
id	Long	Clave primaria.
nombre	String	Identificador comercial (ej. "Pista Central").
tipo	String	Categoría para filtros (ej. "Tenis").
ubicacion	String	Localización física en el campus.
imagenUrl	String	Referencia al archivo multimedia.

Cuadro 4.2: Diccionario de datos: Entidad Espacio Deportivo

4.1.3. Entidad: Reserva

La clase `Reserva` es la entidad asociativa que vincula usuarios y espacios en el tiempo. Es el componente más complejo del modelo debido a la gestión temporal.

Características técnicas:

- **Relaciones:** Implementa relaciones `@ManyToOne` obligatorias (`optional = false`) hacia `Usuario` y `EspacioDeportivo`, garantizando que no existan reservas huérfanas.
- **Auditoría:** Incluye un campo `creadoEn` que se inicializa automáticamente con `LocalDateTime.now()` al instanciar el objeto, permitiendo trazar cuándo se realizó la operación.
- **Patrón View Helper (Adaptadores de Fecha):** Aunque el modelo utiliza la API moderna `java.time.LocalDateTime` para la lógica de negocio, JPA y las vistas JSP (específicamente la librería `JSTL fmt`) tienen problemas de compatibilidad.

Para solucionar esto, la entidad expone métodos de utilidad específicos (`getInicioDate`, `getFinDate`) que convierten los tiempos a `java.util.Date` en tiempo de ejecución. Esto permite usar etiquetas como `<fmt:formatDate>` en el Frontend sin ensuciar la lógica del modelo con tipos de datos obsoletos.

Atributo	Tipo Java	Descripción
id	Long	Clave primaria.
usuario	Usuario	FK: Usuario titular.
espacio	EspacioDeportivo	FK: Instalación reservada.
inicio	LocalDateTime	Fecha/Hora comienzo del slot.
fin	LocalDateTime	Fecha/Hora fin del slot.
creadoEn	LocalDateTime	Timestamp de auditoría.

Cuadro 4.3: Diccionario de datos: Entidad Reserva

Capítulo 5

Implementación y Desarrollo

5.1. Análisis de los Controladores (Servlets)

5.1.1. Controlador de Usuarios

El núcleo de la gestión de usuarios reside en la clase `controladorUsuario`, un Servlet que centraliza la lógica de negocio y la interacción con la capa de persistencia. Este controlador extiende de `HttpServlet` y está decorado con la anotación `@WebServlet`, definiendo un patrón de URL `/usuario/*` que permite capturar múltiples sub-rutas.

Arquitectura y Enrutamiento (Front Controller)

A diferencia de una arquitectura de servlets independientes, este controlador implementa una lógica de enrutamiento interno basada en `getPathInfo()`. Mediante estructuras `switch`, el método `doGet` gestiona la navegación (vistas como `/panel`, `/editar`) y el método `doPost` administra las acciones transaccionales (`/save`, `/aprobarSolicitud`).

Esta estrategia reduce la dispersión del código y centraliza las validaciones de seguridad en un único punto de entrada antes de despachar la petición a los JSPs o ejecutar lógica de negocio.

Gestión Transaccional (JTA y JPA)

Uno de los aspectos técnicos más robustos del controlador es la gestión explícita de transacciones. Se utiliza inyección de dependencias para obtener el `EntityManager` (JPA) y, crucialmente, el recurso `UserTransaction` (JTA).

- **Control de Atomicidad:** Operaciones críticas como `save` o `procesarAprobarSolicitud` están envueltas en bloques `try-catch` donde se ejecutan manualmente `utx.begin()` y `utx.commit()`.
- **Manejo de Fallos:** En caso de excepción durante la persistencia, el sistema ejecuta un `utx.rollback()` controlado para asegurar la integridad de la base de datos, evitando estados inconsistentes.

Lógica de Negocio y Seguridad

El método `procesarGuardarUsuario` implementa una lógica “Upsert” (Update/Insert) inteligente:

1. **Detección de estado:** Verifica si llega un ID para decidir si persistir (`em.persist`) o fusionar (`em.merge`).
2. **Validación de Unicidad:** Antes de escribir en la BD, realiza consultas JPQL personalizadas para asegurar que no existan duplicados de DNI o Email, excluyendo al propio usuario en caso de edición.
3. **Hashing:** Implementa seguridad a nivel de aplicación transformando las contraseñas con el algoritmo MD5 antes de su almacenamiento.

Interoperabilidad

Para soportar la interfaz dinámica, el controlador actúa como una API REST simplificada en los endpoints `/validar-email` y `/validar-dni`. En lugar de redirigir a una vista, estos métodos configuran el tipo de contenido a `application/json` y escriben directamente objetos JSON en el flujo de salida (`PrintWriter`), permitiendo al cliente JavaScript validar campos en tiempo real sin recargar la página haciendo uso de fetch.

El código fuente completo está disponible en: [ControladorUsuario.java](#)

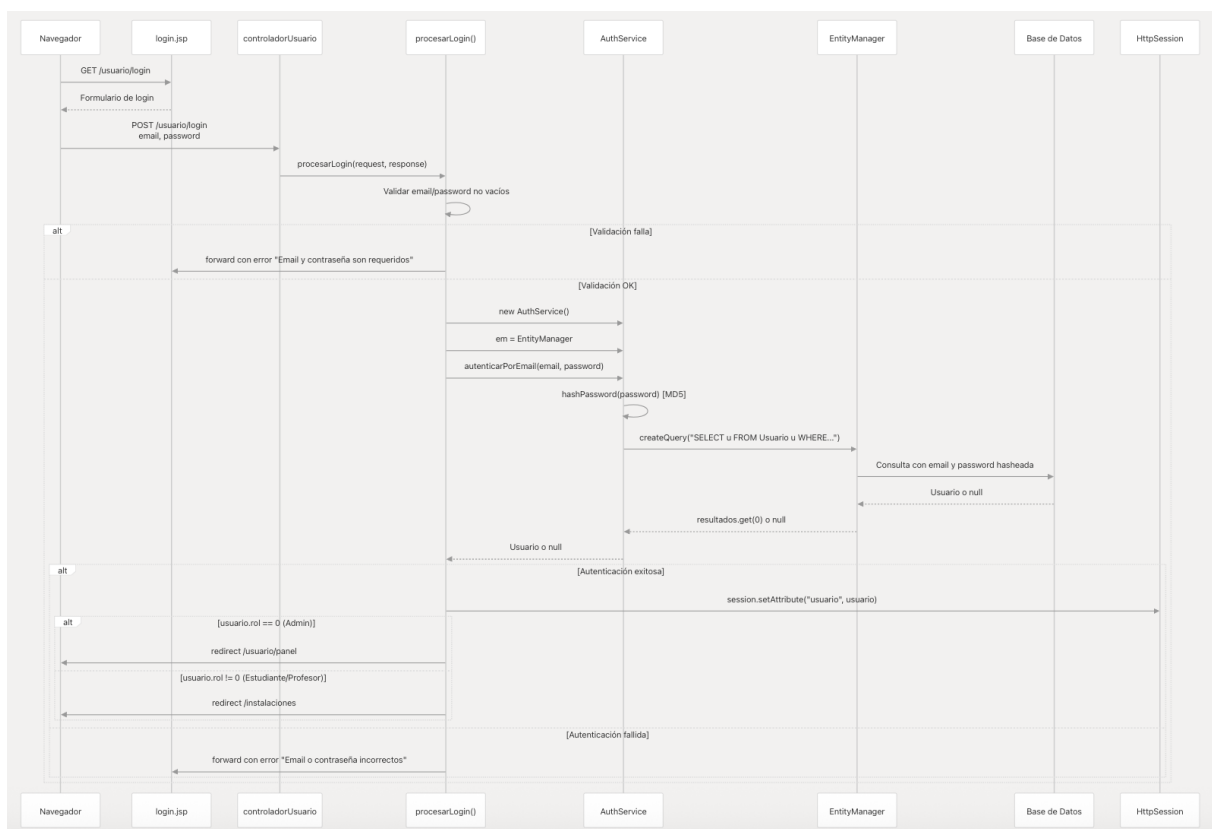


Figura 5.1: Diagrama de secuencia del login

5.1.2. Controlador de Espacios Deportivos

El `ControladorEspacioDeportivo` es un Servlet diseñado para la gestión del inventario de instalaciones. A diferencia de los controladores estándar, este componente está decorado con la anotación `@MultipartConfig`, lo que le habilita para procesar solicitudes complejas que incluyen la transmisión de archivos binarios (imágenes) junto con datos de texto (formularios *multipart/form-data*).

Gestión Avanzada de Archivos Multimedia

Uno de los desafíos técnicos resueltos en este controlador es la persistencia y consistencia de las imágenes de las instalaciones. El método `procesarGuardarInstalacion` implementa un flujo de tratamiento de archivos robusto:

1. **Generación de Identificadores Únicos:** Para evitar colisiones de nombres en el sistema de archivos del servidor, se utiliza `UUID.randomUUID()` para renombrar cada imagen entrante antes de su almacenamiento.
2. **Estrategia de Doble Persistencia:** El controlador detecta las rutas absolutas del servidor en tiempo de ejecución para guardar la imagen en dos ubicaciones simultáneamente:
 - **Directorio de Despliegue (Build):** Permite que la imagen esté disponible inmediatamente para el servidor web activo sin necesidad de reinicio.
 - **Directorio Fuente (Source):** Copia el archivo a la estructura original del proyecto (`web/img`), garantizando que los archivos subidos no se pierdan al redestregar o recompilar la aplicación (persistencia en desarrollo).
3. **Normalización de Rutas:** Se almacenan rutas relativas en la base de datos (`/img/instalaciones/temp/...`) para desacoplar la capa de datos de la estructura física del servidor.

Seguridad y Control Transaccional

Siguiendo el patrón establecido en la arquitectura del sistema, el controlador integra la gestión de transacciones JTA (`UserTransaction`) para asegurar la atomicidad en las operaciones de escritura.

- **Integridad Referencial:** Las operaciones de eliminación (`borrarInstalacion`) y modificación se ejecutan dentro de límites transaccionales estrictos, asegurando que no se eliminen registros si ocurre un fallo en el proceso.
- **Control de Acceso Basado en Roles (RBAC):** Se implementan guardas de seguridad al inicio de cada ruta crítica (`/nueva`, `/editar`, `/borrar`). El método auxiliar `esAdministrador` verifica la sesión HTTP, impidiendo que usuarios no autorizados (Estudiantes o Profesores) accedan a las vistas de gestión o invoquen acciones destructivas, redirigiéndolos a una página de error controlada.

Funcionalidades técnicas destacadas:

- Uso de la API NIO de Java (`java.nio.file.Files`) para operaciones eficientes de sistema de archivos.
- Implementación de patrón *Front Controller* para enrutamiento interno de vistas públicas y privadas.
- Validación de servidor para campos obligatorios y tipos de datos antes de la persistencia.

El código fuente detallado, incluyendo la lógica de manipulación de archivos, se encuentra disponible en: [ControladorEspacioDeportivo.java](#)

5.1.3. Controlador de Reservas

El `ControladorReserva` constituye el núcleo transaccional del sistema. Debido a la naturaleza crítica de las operaciones que maneja (bloqueo de recursos compartidos y transacciones económicas), este componente implementa una lógica de negocio estricta y mecanismos de integración con servicios externos (Stripe API) a bajo nivel.

Algoritmo de Disponibilidad y Calendario

La gestión del tiempo no se basa en una tabla estática de horarios, sino en un algoritmo generativo implementado en tiempo de ejecución:

- **Segmentación Temporal:** El controlador divide dinámicamente el día en *slots* de 90 minutos (constante `DURACION_RESERVA_MINUTOS`), operando en el rango de 08:30 a 20:30.
- **Filtrado Lógico:** Implementa reglas de exclusión para fines de semana mediante la API `java.time.DayOfWeek` y verifica colisiones contra la base de datos utilizando consultas JPQL que detectan solapamientos de intervalos temporales.
- **Respuesta asincronas Optimizada:** Para la vista de calendario interactivo, el controlador construye la respuesta JSON manualmente mediante `StringBuilder` en lugar de utilizar librerías de serialización pesadas, maximizando el rendimiento en la transmisión de datos de disponibilidad.

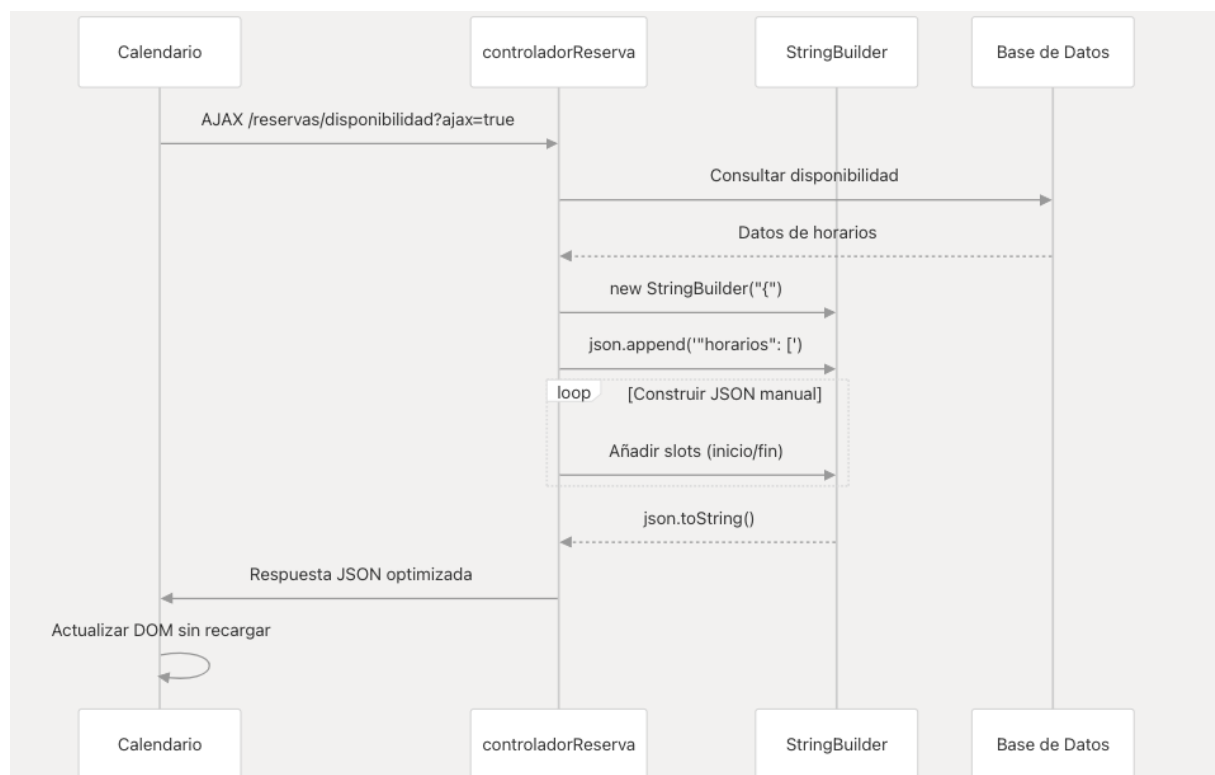


Figura 5.2: Diagrama de secuencia del algoritmo de disponibilidad

Motor de Precios Dinámico

El cálculo de costes se realiza mediante un motor de reglas de negocio hardcodeado en el método `calcularPrecio`. Este motor analiza semánticamente el nombre y descripción de la instalación (buscando palabras clave como "luz", "pabellón", "tenis") y cruza esta información con los atributos del usuario (posesión de tarjeta TUO, rol de Profesor) para determinar el precio final en tiempo real, sin necesidad de tablas de tarifas complejas en la base de datos.

Integración de Pagos y Seguridad Transaccional

El sistema implementa una integración directa con la API de Stripe sin utilizar SDKs de terceros, demostrando un control total sobre la comunicación HTTP:

1. **Cliente HTTP Nativo:** Se utiliza `HttpsURLConnection` para establecer comunicaciones seguras con los endpoints REST de Stripe, gestionando manualmente las cabeceras de autorización y el payload de la transacción.
2. **Prevención de Condiciones de Carrera (Race Conditions):** Para evitar el problema del "doble booking" (dos usuarios reservando la misma pista al mismo milisegundo), el controlador realiza una verificación de disponibilidad de doble fase: una pre-pago y otra post-pago, justo antes del *commit* en base de datos.
3. **Transaccionalidad JTA:** Al igual que el resto del sistema, el ciclo completo (carga en tarjeta + inserción en BD) está protegido por `UserTransaction`.

Detalles de implementación técnica:

- Bypass de validación SSL (`X509TrustManager`) condicional para facilitar el desarrollo en entornos locales sin certificados válidos.
- Uso intensivo de la API `java.time` (`LocalDate`, `LocalTime`, `LocalDateTime`) para aritmética de fechas precisa.
- Parseo de respuestas JSON externas utilizando la API estándar `jakarta.json`.

El código fuente con la integración de la pasarela de pagos y lógica de negocio se encuentra en: [ControladorReserva.java](#)

- **Configuración de entorno local:** Para habilitar la pasarela de pagos, localice la constante `STRIPE_SECRET_KEY` en el controlador. El valor debe completarse añadiendo, justo después del prefijo `sk_`, el *hash* disponible en el archivo de configuración [conf/data](#).

Diagrama de secuencia sobre los pagos con la pasarela

La Figura 5.3 detalla el ciclo de vida completo de una transacción de reserva. El diagrama ilustra la orquestación entre las vistas JSP, el controlador (Backend) y la API externa de Stripe. Es relevante destacar el bloque **alt** de validación, donde se aprecia la estrategia de defensa contra condiciones de carrera (*race conditions*): el sistema verifica la disponibilidad del *slot* temporal una última vez (**Validar colisión**) justo antes de ejecutar la llamada síncrona a la API de Stripe y confirmar la persistencia en la base de datos.

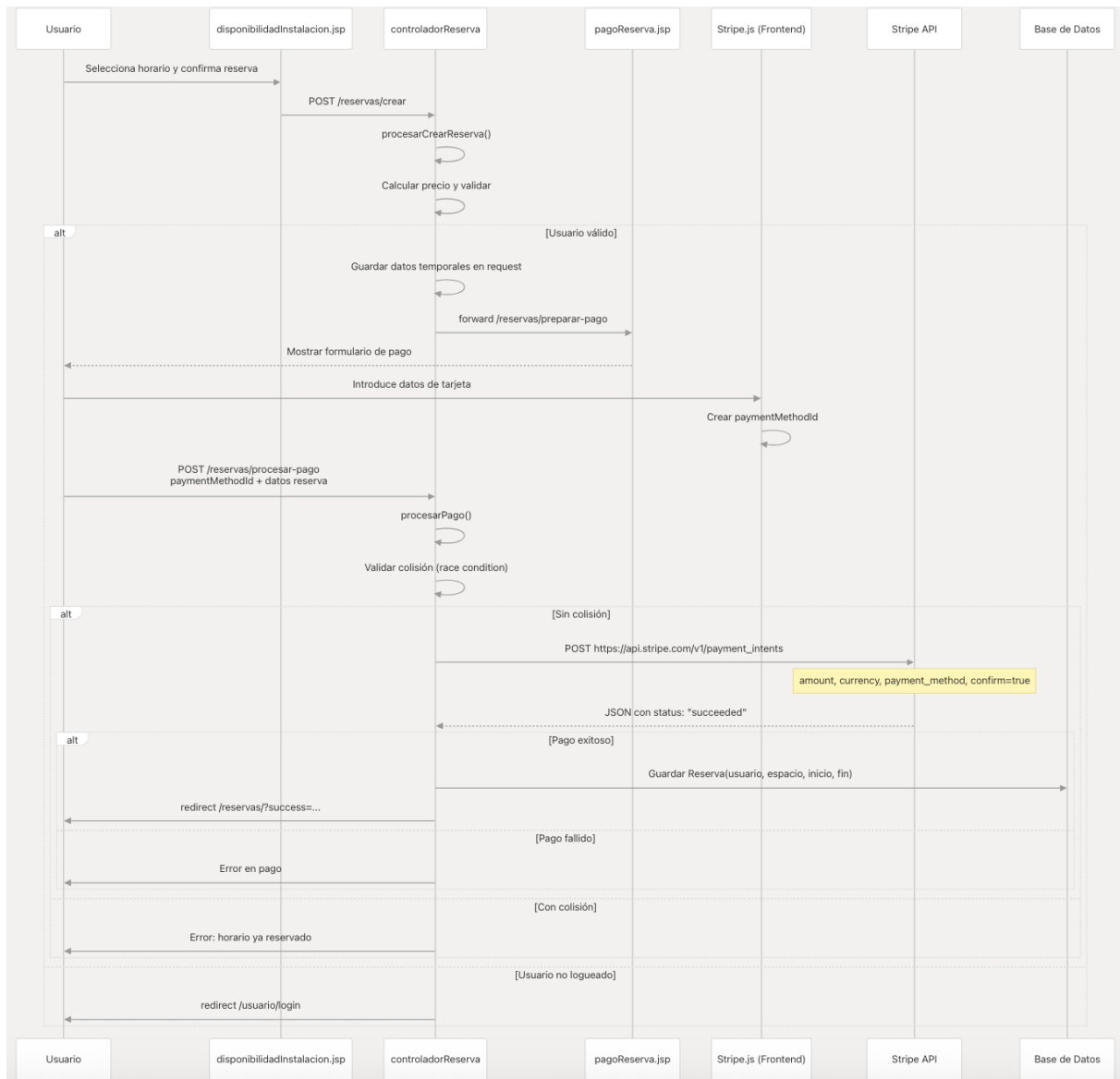


Figura 5.3: Pagos a través de la pasarela (STRIPE)

Capítulo 6

Capa de Presentación

6.1. Arquitectura de Vistas

El sistema implementa un patrón de diseño de Plantilla Maestra (Master Layout) para garantizar la consistencia visual y reducir la duplicidad de código HTML. En lugar de que cada vista JSP contenga la estructura completa del documento (HTML, HEAD, BODY), el sistema utiliza un contenedor principal que inyecta dinámicamente el contenido específico de cada módulo.

6.1.1. Estructura de Directorios

Las vistas se encuentran protegidas bajo el directorio `/WEB-INF/vistas/`, impidiendo el acceso directo por parte del cliente y obligando a que toda navegación pase a través de los Controladores (Servlets). La organización jerárquica es la siguiente:

- `/auth/`: Vistas de acceso público (Login, Registro).
- `/admin/`: Vistas de gestión exclusiva para administradores (Panel de usuarios, Solicitudes).
- `/instalaciones/`: Catálogo visual, detalles y formularios de gestión de espacios.
- `/reservas/`: Calendarios de disponibilidad, formularios de reserva y pasarela de pago.
- `/templates/`: Componentes estructurales y plantillas reutilizables (`layout.jsp`).

6.1.2. Mecanismo de Inyección Dinámica

El núcleo de la presentación reside en el archivo `layout.jsp`. Este fichero define el esqueleto de la aplicación (Cabecera, Navegación, Pie de página, Importación de CSS/JS) y define una "zona de contenido variable".

El flujo de renderizado funciona de la siguiente manera:

1. **Controlador:** Procesa la lógica y define qué vista parcial se debe mostrar guardando su ruta en el atributo `pageContent`.
2. **Despacho:** El controlador redirige siempre al `layout.jsp` mediante `RequestDispatcher`.

3. **Ensamblaje:** El `layout.jsp` recibe la petición y utiliza la etiqueta estándar de JSP para incrustar el contenido:

```
<main class="main-content">
  <c:if test="${not empty pageContent}">
    <jsp:include page="${pageContent}" />
  </c:if>
</main>
```

6.1.3. Lógica de Presentación (JSTL)

Para dotar de inteligencia a las vistas sin introducir código Java sucio (*Scriptlets*), se utiliza extensivamente la librería **Jakarta Standard Tag Library (JSTL)**.

Renderizado Condicional por Roles

La barra de navegación y los botones de acción se adaptan en tiempo real al rol del usuario almacenado en la sesión. El sistema evalúa la propiedad `rol` del objeto usuario para mostrar u ocultar elementos sensibles:

```
<c:if test="${sessionScope.usuario.rol == 0}">
  <a href="../../../editar" class="btn-action">
    <svg>...</svg> Editar
  </a>
  <a href="../../../borrar" class="btn-danger">
    <svg>...</svg> Eliminar
  </a>
</c:if>
```

Feedback al Usuario

El sistema de vistas implementa un mecanismo de notificaciones contextuales. Los controladores inyectan atributos como `msg`, `error` o `success`, y el layout determina cómo mostrarlos (alertas flotantes, modales o mensajes en línea) para informar al usuario del resultado de sus acciones.

6.1.4. Integración de Recursos Estáticos

La gestión de estilos y scripts sigue una estrategia modular para optimizar la carga:

- **CSS Modular:** Además de una hoja de estilos base, el layout carga condicionalmente hojas de estilo específicas (`usuarios.css`, `reservas.css`) dependiendo del módulo activo.
- **JavaScript no intrusivo:** La lógica de cliente (validaciones de formularios, interacción con Stripe, confirmaciones de borrado) se encuentra separada en archivos `.js` dedicados, manteniendo el HTML limpio y semántico.

Capítulo 7

Lógica de Cliente (Scripts)

La interactividad del cliente se gestiona mediante una colección de scripts en **Vanilla JavaScript** (ES6), ubicados en el directorio `/web/scripts/`. Esta capa se encarga de la validación preventiva de datos, la interactividad de la interfaz (UI) y la seguridad en acciones de sesión, sin depender de frameworks externos para minimizar la carga de red.

7.1. Arquitectura de Ficheros

El sistema carga estos recursos de forma condicional o global a través de la plantilla maestra (`layout.jsp`), asegurando que las funciones estén disponibles en todo el ciclo de vida de la aplicación.

Fichero	Responsabilidad Principal
<code>validacion-formularios.js</code>	Validación síncrona (Regex) y asíncrona (fetch).
<code>index.js</code>	Comportamiento global del DOM y UI responsiva.
<code>logout.js</code>	Seguridad y confirmación de cierre de sesión.

Cuadro 7.1: Organización de scripts del cliente

7.2. Análisis de Componentes

7.2.1. Validación Híbrida (`validacion-formularios.js`)

Este script es el componente más complejo del frontend. Implementa un patrón de validación en tiempo real que combina comprobaciones locales con verificaciones en el servidor mediante `fetch` API.

Funcionalidades implementadas:

- **Verificación Asíncrona de Unicidad:** Antes de enviar el formulario de registro o edición, el script lanza peticiones HTTP GET a los endpoints `/usuario/validar-email` y `/usuario/validar-dni`. Esto permite informar al usuario si un correo o DNI ya está registrado sin necesidad de recargar la página.

- **Validación de Formatos (Regex):** Aplica expresiones regulares estrictas para validar el formato del correo electrónico y la robustez de la contraseña (longitud mínima, caracteres obligatorios) al perder el foco del campo (evento `blur`).
- **Feedback Visual:** Manipula el DOM para inyectar mensajes de error o marcas de éxito (clases CSS `.is-invalid` / `.is-valid`) directamente en los inputs afectados.

7.2.2. Gestión de Interfaz (`index.js`)

Actúa como el punto de entrada principal (*entry point*) para la lógica visual de la plantilla. Su función principal es garantizar la usabilidad en dispositivos móviles y gestionar los elementos interactivos comunes.

Lógica destacada:

- **Menú Hamburguesa:** Controla el evento `click` en el botón de navegación móvil para alternar la visibilidad del menú lateral (clase `.show` o similar), permitiendo una experiencia responsiva fluida.
- **Inicialización de Componentes:** Se encarga de detectar y activar elementos de la interfaz que requieren inicialización por JavaScript al cargar el documento (evento `DOMContentLoaded`).

7.2.3. Control de Sesión (`logout.js`)

Un script especializado en la seguridad de la acción de salida. Intercepta el evento de navegación en el enlace de “Cerrar Sesión” para evitar desconexiones accidentales.

Flujo de ejecución:

1. Escucha global de clics en elementos identificados como disparadores de logout.
2. Previene el comportamiento por defecto del enlace (`event.preventDefault()`).
3. Despliega un diálogo de confirmación nativo o modal.
4. Solo si el usuario confirma explícitamente, redirige programáticamente a la ruta `/usuario/logout`, que invoca la invalidación de la sesión en el servidor.

Capítulo 8

Configuración y Despliegue

8.1. Requisitos del Sistema

8.1.1. Requisitos Mínimos

- **Java Development Kit (JDK):** Versión 8 o superior
- **GlassFish Server:** Versión 5.x o superior
- **Memoria RAM:** Mínimo 2GB recomendados
- **Disco Duro:** 500MB de espacio libre
- **Sistema Operativo:** Windows, Linux o macOS

8.1.2. Requisitos Recomendados

- **Java Development Kit (JDK):** Versión 11 LTS
- **GlassFish Server:** Versión 6.x
- **Memoria RAM:** 4GB o más
- **Disco Duro:** 1GB de espacio libre
- **Base de Datos:** MySQL 8.x o PostgreSQL 13.x

8.2. Configuración del Entorno

8.2.1. Configuración NetBeans

El proyecto incluye configuración específica para NetBeans en `nbproject/`:

Listing 8.1: `project.properties` - Configuración principal

```
1 build.dir=build
2 build.web.dir=${build.dir}/web
3 build.classes.dir=${build.dir}/web/WEB-INF/classes
4 build.generated.dir=${build.dir}/generated
5 dist.dir=dist
6 # Configuraci n de GlassFish
7 j2ee.server.domain=domain1
8 j2ee.server.home=${glassfish.home}
9 j2ee.server.instance=localhost
10 j2ee.server.name=GlassFish
11 j2ee.server.port=8080
```

8.2.2. Configuración GlassFish

Listing 8.2: Configuración típica del servidor

```
1 # Puerto HTTP: 8080
2 # Puerto HTTPS: 8181
3 # Puerto Admin: 4848
4 # Context Root: /instalaciones-uhu
5 # Java EE Version: 8
```

8.3. Proceso de Despliegue

8.3.1. Despliegue en Desarrollo

1. Importar proyecto en NetBeans:

- File → Open Project
- Seleccionar carpeta `instalaciones-uhu`
- NetBeans detectará automáticamente la estructura Ant

2. Configurar servidor GlassFish:

- Tools → Servers → Add Server
- Seleccionar GlassFish Server
- Especificar ubicación de instalación
- Configurar dominio (generalmente `domain1`)

3. Compilar proyecto:

- Clean and Build (F11)
- Verificar que no hay errores de compilación

4. Desplegar aplicación:

- Run Project (F6)
- La aplicación se desplegará automáticamente
- Abrir navegador en `http://localhost:8080/instalaciones-uhu`

8.3.2. Despliegue en Producción

1. Generar archivo WAR:

- Clean and Build en NetBeans
- El archivo `instalaciones.war` se genera en `dist/`

2. Desplegar en GlassFish:

- Acceder a consola admin: `https://localhost:4848`
- Applications → Deploy
- Seleccionar archivo `instalaciones.war`
- Especificar contexto si es necesario

3. Verificar despliegue:

- Acceder a `http://servidor:8080/instalaciones-uhu`
- Verificar que todas las funcionalidades trabajen correctamente

8.4. Configuración de Base de Datos

Aunque el código actual no incluye configuración explícita de base de datos, la estructura está preparada para integración con:

- **MySQL:** Mediante connector JDBC
- **PostgreSQL:** Driver PostgreSQL JDBC
- **Apache Derby:** Para desarrollo y testing

La configuración de conexión se manejaría típicamente mediante:

- Archivo `context.xml` en `META-INF/`
- Configuración JNDI en GlassFish
- Pool de conexiones configurado en el servidor

Capítulo 9

Funcionalidades del Sistema

9.1. Módulo de Autenticación y Autorización

9.1.1. Registro de Usuarios

- **Validación de datos:** Email válido, fortaleza de contraseña, campos obligatorios
- **Verificación de unicidad:** Control de emails duplicados
- **Seguridad:** Hash de contraseñas
- **Experiencia de usuario:** Mensajes de error claros, redirección automática

9.1.2. Inicio de Sesión

- **Autenticación segura:** Verificación de credenciales con hash
- **Gestión de sesiones:** Creación y mantenimiento de sesiones HTTP
- **Control de acceso:** Redirección según roles de usuario
- **Seguridad:** Protección contra ataques de fuerza bruta

9.1.3. Gestión de Perfiles

- **Edición de datos:** Actualización de información personal
- **Seguridad:** Verificación de identidad para cambios sensibles
- **Persistencia:** Almacenamiento seguro de modificaciones

9.2. Módulo de Gestión de Instalaciones

9.2.1. CRUD de Espacios Deportivos

- **Crear instalaciones:** Formularios completos con validación
- **Listar instalaciones:** Vista paginada y filtrada
- **Editar instalaciones:** Modificación de datos existentes
- **Eliminar instalaciones:** Borrado lógico o físico

9.2.2. Gestión de Imágenes

- **Upload de archivos:** Soporte para imágenes de instalaciones
- **Validación:** Tipo, tamaño y dimensiones de archivos
- **Almacenamiento:** Gestión de rutas y nombres de archivos
- **Visualización:** Galerías y vistas previas

9.3. Módulo de Reservas

9.3.1. Creación de Reservas

- **Selección de instalación:** Interfaz intuitiva para elegir espacio
- **Selección de fecha/hora:** Calendarios y selectores optimizados
- **Validación de disponibilidad:** Verificación en tiempo real
- **Confirmación:** Resumen y confirmación antes de crear

9.3.2. Gestión de Disponibilidad

- **Verificación en tiempo real:** Comprobación inmediata de conflictos
- **Control de horarios:** Validación de horarios de apertura/cierre
- **Prevención de conflictos:** Detección de solapamientos
- **Actualización dinámica:** Interfaz que refleja disponibilidad actual

9.3.3. Historial y Gestión

- **Listado de reservas:** Vista cronológica de reservas propias
- **Detalles de reserva:** Información completa de cada reserva
- **Cancelación:** Sistema para cancelar reservas futuras
- **Exportación:** Posibilidad de descargar información de reservas

9.4. Panel Administrativo

9.4.1. Gestión de Usuarios

- **Listado completo:** Vista de todos los usuarios registrados
- **Edición avanzada:** Modificación de roles y estados
- **Búsqueda:** Filtrado por diferentes criterios
- **Estadísticas:** Datos de uso y actividad

9.4.2. Administración del Sistema

- **Monitorización:** Estado del sistema y uso de recursos
- **Configuración:** Ajustes generales de la aplicación
- **Backups:** Herramientas para respaldo de datos
- **Logs:** Visualización de registros del sistema

9.5. Características de Seguridad

9.5.1. Protección de Datos

- **Hash de contraseñas:** Uso de MD5 (Debería cambiar para uso en producción)
- **Validación de entrada:** Sanitización de datos del usuario
- **Protección XSS:** Escapado de contenido en vistas
- **Control de sesiones:** Timeout y regeneración de IDs

9.5.2. Control de Acceso

- **Autorización por roles:** Diferenciación usuario/administrador
- **Rutas protegidas:** Acceso restringido según permisos
- **Verificación de sesiones:** Control en cada petición sensible
- **Redirecciones seguras:** Prevención de acceso no autorizado

9.6. Conclusión Final

El proyecto **Instalaciones UHU** representa una implementación sólida y bien estructurada de un sistema de gestión de instalaciones deportivas universitarias. A través del uso de tecnologías Java EE estándar y la aplicación consistente de mejores prácticas de desarrollo, se ha creado una plataforma que satisface los requisitos fundamentales de funcionalidad, seguridad y usabilidad.

La arquitectura MVC implementada proporciona una base excelente para mantenimiento y extensiones futuras. El sistema de autenticación seguro, combinado con un control de acceso basado en roles, garantiza la protección adecuada de los datos y funcionalidades. La interfaz de usuario, aunque simple, es funcional y proporciona una experiencia coherente across diferentes módulos.

Las limitaciones actuales, particularmente en cuanto a persistencia de datos y algunas funcionalidades avanzadas, representan oportunidades claras para futuras iteraciones del proyecto. El código base está bien preparado para acomodar estas mejoras sin requerir reescrituras significativas.

En conjunto, este proyecto demuestra competencia en el desarrollo de aplicaciones web empresariales con Jakarta EE y sienta las bases para un sistema de producción robusto y escalable.